claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-aa0e778fece4ff079.jsonl`
- SHA-256: `6933a942046f63c4c30e18f2bd567f6b6160f7bdcacfe09f5e5792275e55593b`
- Source modified: `2026-07-02T02:53:14+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.979Z)

```
CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: CI + test health of the ENGINE. Repo: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
Check: gh run list --limit 15 (any red/flaky on master?); .github/workflows/*.yml - duplication that issue #384 tracks (shared offline-guard lanes + coverage floor via workflow_call - verify current state), per-job timeouts, nightly.yml health (gh run list --workflow=nightly.yml --limit 10 - is the Tier-1 nightly green or silently red?); pytest config + coverage floor (pyproject/setup.cfg --cov-fail-under vs actual); count of skipped tests and why (grep for skipif/skip markers - are the 12 obsreport skips still the only ones?); slowest-test hygiene;
```

ruff/mypy config drift. Report red/flaky CI as P0/P1.

2. user (2026-07-02T02:47:20.146Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

master...origin/master

3. user (2026-07-02T02:47:21.189Z)

```
.github\workflows\nightly.yml
.github\workflows\ci.yml
```

4. user (2026-07-02T02:47:29.690Z)

```
completed       success api: /api/videos surfaces fps + resolution (from the stored ffprobe d?
CI      master  push    28532862647     2m58s   2026-07-01T16:36:43Z
completed       success api: /api/videos returns video_id + a friendly match_name (fix disapp?
CI      master  push    28531005211     2m53s   2026-07-01T16:04:25Z
completed       success docs: reflect today's storage endpoints + the corpus/user-video bound?
CI      master  push    28530352910     2m40s   2026-07-01T15:53:37Z
completed       success deploy/gcp: centralize project + bucket names in one sourced buckets.?
CI      master  push    28528006637     3m0s    2026-07-01T15:15:59Z
completed       success P4: POST /api/assets ? store a video into the user's chosen medium (#?
CI      master  push    28524662552     2m45s   2026-07-01T14:23:42Z
completed       success P3: /api/capabilities reports usable storage backends (#408)    CI
master  push    28523306298     2m50s   2026-07-01T14:02:36Z
completed       success P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
CI      master  push    28523034384     2m37s   2026-07-01T13:58:28Z
completed       success feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
CI      master  push    28519443869     2m48s   2026-07-01T13:01:41Z
completed       success feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in?
CI      master  push    28503170582     2m49s   2026-07-01T08:09:11Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28499884740     2m48s
2026-07-01T07:05:07Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28425848895     2m42s
2026-06-30T06:49:07Z
completed       success docs: record the launch NO-GO decision (owner, 2026-06-30) (#404)
CI      master  push    28419282606     2m56s   2026-06-30T03:59:24Z
completed       success docs: fix stale rally-cluster claims (no archival ? cluster still gat?
CI      master  push    28417968091     2m37s   2026-06-30T03:20:56Z
completed       success chore(#399): launch-PR process hardening ? link-check CI, PR template?
CI      master  push    28417781155     2m31s   2026-06-30T03:15:23Z
completed       success fix(#398): served Gate-A litmus passes a typed IndexingConfig (not a ?
CI      master  push    28417374248     2m52s   2026-06-30T03:03:56Z
```

5. user (2026-07-02T02:47:31.069Z)

```
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28499884740     2m48s
2026-07-01T07:05:07Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28425848895     2m42s
2026-06-30T06:49:07Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28355510707     2m37s
2026-06-29T07:22:43Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28314418544     2m45s
2026-06-28T06:56:14Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
```

```
regression (Tier-1, offline)    master  schedule        28281089675     2m45s
2026-06-27T06:23:13Z
completed       success Nightly quality regression (Tier-1, offline)    Nightly quality
regression (Tier-1, offline)    master  schedule        28222002166     2m30s
2026-06-26T06:44:51Z
```

6. user (2026-07-02T02:47:31.094Z)

```yaml
name: CI

on:
  push:
    branches: [master]
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
  # tracked .py, including scripts/ and training/, which the backend-only
  # py_compile smoke sweep does not reach.
  lint:
    name: Ruff lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github

  # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
  # (the video_id shape) or a personal absolute user path in the shipped-code surface
  # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures). Real
  # names are scanned only via a gitignored private list locally; CI runs the structural guard.
  leak-scan:
    name: Leak scan (PII / attribution guard)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Leak scan
        run: python -m tools.leak_scan

  # Internal-link guard (tools/check_md_links.py): fails on a broken internal markdown link
  # (missing file or dead #anchor) in a LIVE doc. docs/archives/** is excluded ? frozen
  # snapshots whose relative links are historical-by-design (see docs/DOC_STATUS.md §2).
  link-check:
    name: Markdown link check (live docs)
```

```
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Check internal markdown links
        run: python -m tools.check_md_links

  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
  # per-module quarantine list ? ratchet by removing entries as modules clean
  # up. Typed core deps are installed so mypy sees their real signatures
  # (without them ignore_missing_imports would Any-stub half the checks).
  typecheck:
    name: Mypy (lenient baseline)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install mypy + typed core deps
        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai numpy
      - name: Mypy
        run: mypy backend training

  # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
  # module-level import error shipping while the suite stays green.
  # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
  # on machines without the GPU stack (test_import_smoke stubs what's absent).
  import-smoke:
    name: Syntax sweep + import smoke (no GPU stack)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install minimal deps
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
      - name: Run import smoke tests
        run: python -m pytest tests/test_import_smoke.py -v

  full-suite:
    name: Full test suite (offline, mocked)
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
          cache: pip
      - name: System libs for opencv-python
        run: |
          # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it periodically
          # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing the
          # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
```

```
archive.
          sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
          sudo apt-get update
          sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y libgl1 libglib2.0-0
      - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
        # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
        # need an out-of-band index that PEP 621 can't express. Keep installing
        # them separately here, before the editable install pulls the rest.
        run: python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
      - name: Install trackers (git-only dep ? PyPI wheel is broken)
        # D13/P12: ByteTrack moved out of `supervision` (removed in 0.30.0) to the
        # standalone `trackers` package. Its PyPI wheel ships metadata only (no
        # source ? roboflow/trackers#474), so it MUST come from git. PEP 621 can't
        # express a git URL, so it's not in pyproject.toml (same pattern as torch).
        # Installed here so the two D13/P12 tests in tests/test_person_detector.py
        # actually run in CI instead of skipping via pytest.importorskip.
        # NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
        run: python -m pip install "trackers @ git+https://github.com/roboflow/trackers.git@2.5.0"
      - name: Install package (editable) + dev tooling
        # Editable PEP 621 install: runtime deps + the `dev` group
        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
        run: python -m pip install -e .[dev]
      # obsreport (private, soft dep) is absent here: reporting tests skip via
      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
      # is therefore calibrated against THIS lane's number (local runs with
      # obsreport installed measure a little higher).
      # Floor calibrated 2026-06-11 against this lane's measured 72% (local
      # with obsreport: 73%). Ratchet it UP as coverage grows; never down
      # without an owner decision.
      - name: Run full suite (with coverage floor)
        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70

  # Cross-OS determinism guard for the committed golden regression fixtures
  # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
  # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
  # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
  # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
  # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
  golden-crossos:
    name: Golden fixtures cross-OS (${{ matrix.os }})
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
    runs-on: ${{ matrix.os }}
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai numpy
      - name: Golden-fixtures characterization (cross-OS determinism guard)
        # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests (synthetic /
        # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and the
        # de-attribution paths get cross-OS coverage and surface any win/mac float drift.
```

```yaml
        run: |
          python -m backend.eval.golden_fixtures --check
          python -m pytest tests/test_golden_fixtures.py tests/test_golden_real_fixtures.py -q

  # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d). Scans the
  # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any copyleft
  # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this closure. This
  # automates a guardrail that was human-only; the report is printed for review either way.
  # D13/P12 carve-out: `trackers` (Apache-2.0; ByteTrack's new home) is also git-only and thus
  # absent from this pyproject closure ? UNguarded here, same shape as torch. It's Apache-2.0 and
  # depends only on the still-declared `supervision` (MIT), so the closure is clean today; if
  # `trackers` ever pulls a new transitive dep, this gate won't catch it ? audit on bump.
  license-scan:
    name: Dependency license scan (no copyleft in runtime deps)
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install runtime deps + pip-licenses
        run: |
          python -m pip install .
          python -m pip install pip-licenses
      - name: Report dependency licenses
        run: python -m piplicenses --format=markdown --with-urls --order=license
      - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
        run: |
          if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
            echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL) license ? the guardrail is MIT/Apache/BSD only. Review the report above."
            exit 1
          fi
          echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."

  # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable from a
  # non-repo cwd, and launch the console script. Catches packaging regressions the source suite
  # can't ? namespace-package resolution (backend has no __init__.py), missing data files, hidden
  # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer --help`.)
  build-artifact:
    name: Built wheel smoke (install + launch)
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: System libs for opencv-python
        run: |
          # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
```

```
periodically
          # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing the
          # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's archive.
          sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
          sudo apt-get update
          sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y libgl1 libglib2.0-0
      - name: Build the wheel
        run: |
          python -m pip install build
          python -m build --wheel
      - name: Install the built wheel + CPU torch (out-of-band index)
        run: |
          python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
          python -m pip install dist/*.whl
      - name: Launch the installed console script from a non-repo dir
        run: |
          cd /tmp
          indexer --help
          python -c "import backend, training, backend.utils.app_home; print('wheel import OK')"

```

7. user (2026-07-02T02:47:31.095Z)

```
name: Nightly quality regression (Tier-1, offline)

# Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;
# this runs the SAME offline guards on a schedule so we also catch what per-PR CI
# can't: dependency / transitive-dep drift, flakiness over time, and rot during
# quiet periods when nothing is pushed.
#
# Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the
# real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +
# the gitignored golden corpus).
#
# Lane parity with ci.yml: every OFFLINE guard job from ci.yml is mirrored here
# (lint · leak-scan · typecheck · import-smoke · full-suite · golden-fixtures ·
# license-scan · build-artifact), with the same Python pin + per-lane pip
# installs so modules collect/run instead of skipping for a missing dep. Two
# intentional differences, NOT drops:
#   - golden-fixtures runs single-OS here (ci.yml::golden-crossos already covers
#     the Win/mac/Linux matrix on every PR; the nightly is about drift-over-time,
#     not cross-OS float drift).
#   - a notify-on-failure job (below) surfaces a SCHEDULED failure as a tracking
#     issue ? ci.yml doesn't need it (a PR/push run already has a red check).
# Lane set + coverage floor are still copied literals, not single-sourced ? see
# the `workflow_call` single-sourcing follow-up in the PR description.
#
# NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
# won't actually run on a cadence until it lands on `master`. Use the
# `workflow_dispatch` trigger to run it on demand for validation after merge.

on:
  schedule:
    # 03:00 UTC daily.
    - cron: "0 3 * * *"
  workflow_dispatch:

permissions:
  contents: read

```

```
concurrency:
  group: nightly-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
  lint:
    name: Nightly · Ruff lint
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github

  # PII / attribution leak guard (tools/leak_scan.py) ? mirrors ci.yml::leak-scan.
  # A structural-rot guard: fails on a committed 32-hex MD5 (the video_id shape)
  # or a personal absolute user path in the shipped-code surface.
  leak-scan:
    name: Nightly · Leak scan (PII / attribution guard)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Leak scan
        run: python -m tools.leak_scan

  # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
  # deps are installed so mypy sees real signatures (without them
  # ignore_missing_imports would Any-stub half the checks).
  typecheck:
    name: Nightly · Mypy (lenient baseline)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install mypy + typed core deps
        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai numpy
      - name: Mypy
        run: mypy backend training

  # Syntax sweep + import smoke (no GPU stack) ? mirrors ci.yml::import-smoke.
  # Guards the "SyntaxError / module-level import error shipping while the suite
  # stays green" class. Deliberately does NOT install cv2/torch/numpy.
  import-smoke:
    name: Nightly · Syntax sweep + import smoke (no GPU stack)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
```

```
      - name: Install minimal deps
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
      - name: Run import smoke tests
        run: python -m pytest tests/test_import_smoke.py -v

  # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
  # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
  # rest. Floor stays at the ci.yml value (70); never lower it without an owner
  # decision.
  full-suite:
    name: Nightly · Full test suite (offline, mocked)
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
          cache: pip
      - name: System libs for opencv-python
        run: |
          # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it periodically
          # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing the
          # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's archive.
          sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
          sudo apt-get update
          sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y libgl1 libglib2.0-0
      - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
        run: python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
      - name: Install trackers (git-only dep ? PyPI wheel is broken)
        # D13/P12: ByteTrack moved to the standalone `trackers` package (git-only;
        # see ci.yml for the full rationale). Installed here so the D13/P12 tests
        # run instead of skipping via pytest.importorskip. Bare version tag (no 'v').
        run: python -m pip install "trackers @ git+https://github.com/roboflow/trackers.git@2.5.0"
      - name: Install package (editable) + dev tooling
        run: python -m pip install -e .[dev]
      - name: Run full suite (with coverage floor)
        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70

  # Offline quality guard: the committed golden regression fixtures
  # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
  # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
  # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
  # every PR ? nightly is about drift-over-time on the default branch). The
  # CONTROL replay needs only numpy.
  golden-fixtures:
    name: Nightly · Golden fixtures regression (offline quality guard)
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai numpy
```

```yaml
      - name: Golden-fixtures characterization (offline)
        run: |
          python -m backend.eval.golden_fixtures --check
          python -m pytest tests/test_golden_fixtures.py tests/test_golden_real_fixtures.py -q

  # Commercial-clean licensing guardrail (MIT/Apache/BSD only) ? mirrors
  # ci.yml::license-scan. Scans the RUNTIME dependency closure and fails on any
  # copyleft (GPL/AGPL/LGPL/SSPL). This is exactly the transitive-dep drift the
  # header cites as a reason the nightly exists ? a new sub-dep can flip a
  # license between PRs without touching our code.
  license-scan:
    name: Nightly · Dependency license scan (no copyleft in runtime deps)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: Install runtime deps + pip-licenses
        run: |
          python -m pip install .
          python -m pip install pip-licenses
      - name: Report dependency licenses
        run: python -m piplicenses --format=markdown --with-urls --order=license
      - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
        run: |
          if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
            echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL) license ? the guardrail is MIT/Apache/BSD only. Review the report above."
            exit 1
          fi
          echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."

  # Built-wheel smoke (install + launch) ? mirrors ci.yml::build-artifact. Builds
  # the wheel, installs it NON-editable from a non-repo cwd, and launches the
  # console script. Catches packaging regressions the source suite can't.
  build-artifact:
    name: Nightly · Built wheel smoke (install + launch)
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
      - name: System libs for opencv-python
        run: |
          # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it periodically
          # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing the
          # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's archive.
          sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
          sudo apt-get update
          sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y libgl1 libglib2.0-0
      - name: Build the wheel
        run: |
          python -m pip install build
          python -m build --wheel
      - name: Install the built wheel + CPU torch (out-of-band index)
        run: |
```

```
          python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
          python -m pip install dist/*.whl
      - name: Launch the installed console script from a non-repo dir
        run: |
          cd /tmp
          indexer --help
          python -c "import backend, training, backend.utils.app_home; print('wheel import OK')"

  # Failure visibility ? a silent nightly is useless. Scheduled runs have no PR
  # to redden and nobody is watching the Actions tab on a quiet day (the exact
  # "rot during quiet periods" this workflow exists to catch). On ANY guard
  # failure, open/refresh a single tracking issue so a human is pinged. Uses the
  # built-in GITHUB_TOKEN (no secrets); `issues: write` is scoped to THIS job
  # only ? the guard jobs above keep `contents: read`. Gated on event_name ==
  # 'schedule': a manual `workflow_dispatch` run is being watched, so no issue.
  notify-on-failure:
    name: Nightly · Surface failure (open/refresh tracking issue)
    needs: [lint, leak-scan, typecheck, import-smoke, full-suite, golden-fixtures, license-scan, build-artifact]
    if: failure() && github.event_name == 'schedule'
    runs-on: ubuntu-latest
    timeout-minutes: 5
    permissions:
      contents: read
      issues: write
    steps:
      - uses: actions/github-script@v7
        with:
          script: |
            const runUrl = `${context.serverUrl}/${context.repo.owner}/${context.repo.repo}/actions/runs/${context.runId}`;
            const title = 'Nightly quality regression FAILED';
            const body = [
              'The Tier-1 nightly offline quality guard failed.',
              '',
              `- Run: ${runUrl}`,
              `- Date (UTC): ${new Date().toISOString()}`,
              '',
              'One of ruff / leak-scan / mypy / import-smoke / full-suite (cov>=70) /',
              'golden-fixtures / license-scan / build-artifact regressed.',
              'Open the run for the failing lane. Close this issue once green.',
            ].join('\n');
            // Reuse a single open tracking issue instead of spamming one per night.
            const existing = await github.rest.issues.listForRepo({
              owner: context.repo.owner,
              repo: context.repo.repo,
              state: 'open',
              labels: 'nightly-failure',
              per_page: 1,
            });
            if (existing.data.length > 0) {
              await github.rest.issues.createComment({
                owner: context.repo.owner,
                repo: context.repo.repo,
                issue_number: existing.data[0].number,
                body,
              });
            } else {
              await github.rest.issues.create({
                owner: context.repo.owner,
                repo: context.repo.repo,
                title,
                body,
```

```
                    labels: ['nightly-failure'],
                  });
                }

```

8. user (2026-07-02T02:48:00.027Z)

{"body":"## Follow-up from #381 review (LOW ? drift)\n\n`nightly.yml` mirrors `ci.yml`'s offline guard lanes by **copying** them, and the coverage floor `70` is a hardcoded literal in both workflows. Nothing enforces that they stay in sync ? `ci.yml` has already recalibrated that floor once, and the next ratchet-up would silently leave the nightly on the stale value.\n\n### Proposed fix\nExtract the shared **offline** guard lanes into a reusable workflow ? e.g. `.github/workflows/_offline-guards.yml` exposing them via `on: workflow_call` (with the coverage floor as an input) ? and have **both** `ci.yml` and `nightly.yml` `uses:` it. That single-sources:\n- the lane set (lint · leak-scan · typecheck · import-smoke · full-suite · golden-fixtures · license-scan · build-artifact), and\n- the coverage floor (one `cov-fail-under` value, passed as an input).\n\n### Why deferred from #381\nThe refactor has to modify `ci.yml` as well, which would break #381's single-file scope. Tracked here per the reviewer's \"fine to ship as-is + file a tracked follow-up\" call.\n\n### Scope / non-goals\n- Offline (Tier-1) lanes only. No GPU / real-video / `run_regression.py`.\n- Keep the two intentional nightly-vs-ci differences: nightly's `golden-fixtures` is single-OS (ci's `golden-crossos` covers the matrix per PR), and nightly's `notify-on-failure` job has no ci equivalent.\n\nRefs: #381, `docs/GOLDEN_REGRESSION_FIXTURES.md`.\n","state":"OPEN","title":"ci: single-source the shared offline guard lanes + coverage floor via a workflow_call reusable","url":"https://github.com/Khelsutra/badminton-highlight-indexer/issues/384"}

9. user (2026-07-02T02:48:00.052Z)

```
# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
#
# Build backend: hatchling. `backend/` is an intentional NAMESPACE package
# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
# reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
# auto-detection would otherwise miss the un-__init__'d `backend` root.
#
# torch/torchvision are deliberately NOT listed in [project.dependencies]:
# their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
# which PEP 621 cannot express. Install them separately ? see [project.optional-dependencies]
# `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]

# Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0",
```

```toml
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
    # D13/P12: ByteTrack moved to the standalone `trackers` package (Apache-2.0). Its
    # PyPI wheel is broken (metadata-only), so it MUST be installed from git ? see
    # requirements.txt. PEP 621 cannot express a git URL, so it's NOT listed here
    # (same pattern as torch/torchvision). CI installs it on its own line.
]

[project.optional-dependencies]
# Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]

# M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_auth_enabled=True.
# `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/multi-tenant.
auth = ["PyJWT[crypto]>=2.8"]

# Per-video observability reports (SOFT dependency: the pipeline degrades to a
# no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
# fetched over SSH; if you lack access, simply omit this group ? reporting
# tests skip via pytest.importorskip. For local dev against a sibling checkout:
#   pip install -e ../sports-obsreport --no-build-isolation
reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]

# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
# express, so they are NOT declarable here. Install them out-of-band instead:
#   # CUDA 12.4 box:
#   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org/whl/cu124
#   # CPU-only box (also what CI uses):
#   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
gpu = []

# Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
# these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
# install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 +
# DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
# MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storage/gdrive.py.
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
indexer = "backend.main:main"

[tool.hatch.metadata]
# The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
```

```
# reference*); hatchling requires opting in to allow those in metadata.
allow-direct-references = true

[tool.hatch.build.targets.wheel]
# `backend` is a namespace package (no __init__.py); list both import roots
# explicitly so an editable/wheel install exposes `import backend` and
# `import training` regardless of cwd.
packages = ["backend", "training"]

```

10. user (2026-07-02T02:48:00.421Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""

from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
    bar = "=" * 72
    print(bar)
    print(" Full test suite:  " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)
```

```
    rc = subprocess.run(cmd).returncode

    print(bar)
    print(
        " RESULT: "
        + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
    )
    print(bar)
    return rc


if __name__ == "__main__":
    raise SystemExit(main())

```

11. user (2026-07-02T02:48:18.375Z)

```
pytest.ini
.coveragerc
mypy.ini
ruff.toml
```

12. user (2026-07-02T02:48:19.788Z)

```
tests\test_api_cloud_dispatch.py:1
tests\test_async_jobs.py:1
tests\test_app_home.py:1
tests\test_api_annotations.py:1
tests\test_api_assets.py:1
tests\test_api_endpoints.py:2
tests\test_ablation.py:6
tests\test_bind_and_host.py:1
tests\test_config_deployment.py:1
tests\test_edge_auth.py:2
tests\test_import_smoke.py:2
tests\test_job_worker_loop.py:1
tests\test_nightly_reelcount.py:1
tests\test_person_detector.py:3
tests\test_rally_seg_eval.py:1
tests\test_reporting.py:1
tests\test_reporting_harvest.py:1
tests\test_reporting_spec_faq.py:1
tests\test_served_gate_a_regression.py:2
tests\test_ui_bundle.py:1
tests\test_wasb_cache.py:5
tests\test_upload_download.py:1

Found 37 total occurrences across 22 files.
```

13. user (2026-07-02T02:48:25.094Z)

```
[pytest]
# When `pytest` is invoked without an explicit path, only collect from tests/.
# scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
# and should never run as part of the regular test suite.
testpaths = tests

```

14. user (2026-07-02T02:48:25.550Z)

```
[run]
# Excluded from coverage: the legacy motion segmenter (demo baseline with
# fabricated metadata) and the concrete validators (thin OpenCV/ffprobe
```

```
# wrappers only exercisable against real media files). validators/base.py
# stays counted ? it has a dedicated unit suite.
omit =
    backend/pipeline/segmenters/motion.py
    backend/pipeline/validators/format.py
    backend/pipeline/validators/resolution_fps.py
    backend/pipeline/validators/pts_continuity.py
    backend/pipeline/validators/scene_cut.py
    backend/pipeline/validators/blur.py
    backend/pipeline/validators/relevance.py

```

15. user (2026-07-02T02:48:26.505Z)

```
# Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
# bury blame history for zero behavior value).
#
# Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
# warnings (W) + bugbear (B) + isort (I ? consistent import ordering).
target-version = "py38"

# scratch/ = throwaway debugging scripts that may hit real APIs; it is also
# excluded from pytest collection (see pytest.ini).
extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B", "I"]

[lint.per-file-ignores]
# Entrypoint scripts deliberately run setup (load_dotenv, logging config,
# sys.path) before importing backend modules ? import order is load-bearing.
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]

```

16. user (2026-07-02T02:48:26.814Z)

```
[mypy]
# Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean
# modules and pins an explicit quarantine list below ? not a typing crusade.
# Ratchet: when you touch a quarantined module, try removing its entry.
ignore_missing_imports = True
# backend/ is a namespace package (no root __init__.py) ? required so mypy
# maps files to backend.* module names instead of duplicating top-level names.
explicit_package_bases = True

# --- Quarantine (known-loose modules, each with a reason) ---

# Optional module globals (db_instance/global_config set by create_app() / main()).
# Route handlers use request.app.state exclusively; helpers take explicit params.
# backend.api.job_worker lifted from quarantine (now takes explicit config/db params).
[mypy-backend.main]
ignore_errors = True

# Worker dispatcher: composes the SAME validator/segmenter/report building blocks as
# backend.main, which are typed `config: dict` but receive the AppConfig migration shim
# at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
# until the typed-config migration removes the dict?AppConfig duality everywhere.
[mypy-backend.worker.__main__]
ignore_errors = True

# yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
# to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
# here is pre-existing orchestration debt unrelated to that move (implicit-Optional
```

```
# process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
# annotation) ? ratchet those out in a focused typing pass, not here.
# (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True

# Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True

```

17. user (2026-07-02T02:48:47.345Z)

```
tests\test_ablation.py:400:    pytest.importorskip("reportlab")
tests\test_ablation.py:401:    pypdf = pytest.importorskip("pypdf")
tests\test_ablation.py:525:@pytest.mark.skipif(
tests\test_ablation.py:555:@pytest.mark.skipif(
tests\test_ablation.py:629:    pytest.importorskip("reportlab")
tests\test_ablation.py:630:    pytest.importorskip("pypdf")
tests\test_api_endpoints.py:11:pytest.importorskip("httpx")  # TestClient needs httpx; skip
cleanly where absent
tests\test_api_endpoints.py:889:    pytest.importorskip("obsreport")  # the report writer is a
no-op without it
tests\test_api_assets.py:14:pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly
where absent
tests\test_api_annotations.py:7:pytest.importorskip("httpx")  # TestClient needs httpx; skip
cleanly where absent
tests\test_async_jobs.py:20:pytest.importorskip("httpx")  # TestClient needs httpx; skip cleanly
where absent
tests\test_app_home.py:193:        pytest.skip("Windows drive-letter semantics only meaningful
on nt")
tests\test_api_cloud_dispatch.py:16:pytest.importorskip("httpx")
tests\test_bind_and_host.py:5:pytest.importorskip("httpx")  # TestClient needs httpx
tests\test_config_deployment.py:401:        pytest.skip("no repo config.json")
tests\test_edge_auth.py:14:pytest.importorskip(
tests\test_edge_auth.py:218:pytest.importorskip("httpx")  # fastapi TestClient transport
tests\test_import_smoke.py:110:    pytest.importorskip("numpy")
tests\test_import_smoke.py:111:    pytest.importorskip("scipy")
tests\test_job_worker_loop.py:26:pytest.importorskip("httpx")
tests\test_nightly_reelcount.py:276:    pytest.importorskip("cv2")
tests\test_person_detector.py:215:    pytest.importorskip("trackers")
tests\test_person_detector.py:270:    pytest.importorskip("trackers")
tests\test_person_detector.py:290:    pytest.importorskip("trackers")
tests\test_rally_seg_eval.py:124:    pytest.importorskip(
tests\test_reporting.py:80:# (see test_reporting_harvest.py which uses pytest.importorskip).
Those tests
tests\test_reporting_spec_faq.py:14:pytest.importorskip("obsreport")
tests\test_reporting_harvest.py:11:pytest.importorskip("obsreport")
tests\test_served_gate_a_regression.py:47:pytest.importorskip("cv2")
tests\test_served_gate_a_regression.py:82:_skip = pytest.mark.skipif(
tests\test_ui_bundle.py:8:pytest.importorskip("httpx")
tests\test_upload_download.py:16:pytest.importorskip("httpx")  # TestClient needs httpx; skip
cleanly where absent
tests\test_wasb_cache.py:368:    cv2 = pytest.importorskip("cv2")  # noqa: F841 (importorskip
gates the suite on cv2)
tests\test_wasb_cache.py:375:    cv2 = pytest.importorskip("cv2")
tests\test_wasb_cache.py:401:    pytest.importorskip("cv2")
tests\test_wasb_cache.py:418:    cv2 = pytest.importorskip("cv2")
tests\test_wasb_cache.py:419:    Image = pytest.importorskip("PIL.Image")
```

18. user (2026-07-02T02:48:59.066Z)

```
            granularity="group",
        ),
    )
    out = ablation.format_table(report)
    assert "Ablation report" in out
    assert "?F1(full?ablated)" in out
    assert "verdict" in out
    # the highest-?F1 signal name appears before the lowest in the rendered order
    top = report.sorted_rows()[0].signal
    bottom = report.sorted_rows()[-1].signal
    assert out.index(top) < out.index(bottom)


def test_export_pdf_is_valid_nontrivial(tmp_path):
    """A valid PDF with at least one page is produced, and it contains the verdict legend."""
    pytest.importorskip("reportlab")
    pypdf = pytest.importorskip("pypdf")
    videos = _learned_videos(6)
    report = run_ablation(
        videos,
        AblationConfig(
            full=ablation.learned_full_variant(),
            signals=ablation.default_signals(),
            granularity="group",
        ),
    )
    out = str(tmp_path / "ablation.pdf")
    ablation.export_pdf(report, out)
    assert os.path.isfile(out)
    assert os.path.getsize(out) > 1000  # non-trivial
    reader = pypdf.PdfReader(out)
    assert len(reader.pages) > 0
    text = "".join(p.extract_text() for p in reader.pages)
    assert "Ablation" in text
    assert "clears-bar" in text  # the verdict key rendered


# ---------------------------------------------------------------------- corpus loader (no GPU)


def test_load_videos_tolerates_missing_audio(tmp_path):
    """An older feature JSON without an ``audio`` block still loads ? audio columns default to
    0.0 (loader never crashes on a pre-audio JSON)."""
    cand = {
        "start": 0.0,
        "end": 10.0,
        "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
        "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
        "label": 1,
    }
    data = {
        "name": "noaudio",
        "fps": 30.0,
        "frame_width": 1920.0,
        "candidates": [cand],
        "gts": [[0.0, 10.0]],
    }
    with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
        json.dump(data, f)
    videos = ablation.load_videos(str(tmp_path))
```

19. user (2026-07-02T02:49:00.459Z)

```
                cands = json.load(f).get("candidates") or []
        except (OSError, ValueError):
            continue
        if _is_fusion_schema(
            cands
        ):  # canonical predicate (SSOT in backend.eval.experiment, #215)
            n += 1
    return n >= 6


@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured Gate-A
    result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within tolerance.
    This validates the whole framework against the recorded ground truth. Skips cleanly in CI
    where the golden substrate is absent; runs on the owner box where it is present."""
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) == 6
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
    assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
    assert row.sign_wins == 6 and row.sign_losses == 0
    assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
    assert row.ci_low == pytest.approx(0.042, abs=0.01)
    assert row.ci_high == pytest.approx(0.104, abs=0.01)
    assert row.ci_excludes_zero is True
    # Gate-A is structural ? reported but never auto-promoted to earns_keep.
    assert row.verdict == ablation.STRUCTURAL_PROTECTED
    # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
    assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
    assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)


@pytest.mark.skipif(
    not _golden_present(), reason="golden output/*_golden_features.json absent"
)
def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
    assert isinstance(report, AblationReport)
    assert report.num_videos == 6
    assert report.full_set_signals == ["gate_a"]
    assert report.label_set_hash
    out = ablation.format_table(report)
    assert "gate_a" in out


# ------------------------------------------------------------------------------- edge cases


def test_min_players_gate_knob_zeroed():
    """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
    full = Variant(name="full", min_players=2, feature_names=["a"])
```

20. user (2026-07-02T02:49:01.408Z)

```
import json
import os

import pytest

# cv2 is imported transitively by trajectory_hybrid (the served engine). On a box without it
# (CI), skip the whole module rather than erroring at import.
pytest.importorskip("cv2")

from backend.eval import segmentation_metrics as _segm  # noqa: E402
from backend.eval import served_gate_a as sga  # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts  # noqa: E402
from backend.eval.metrics import score  # noqa: E402
from backend.pipeline.detectors import rally_gate  # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner  # noqa: E402

# The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh
# worktree / CI). Resolve it relative to this repo root first, with an env override for an
# out-of-tree checkout.
_REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MANIFEST = os.environ.get(
    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
)


def _golden_specs():
    """Return the manifest specs whose trajectory + label files are all present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
    with open(MANIFEST, encoding="utf-8") as f:
        specs = json.load(f)
    ok = [
        s
        for s in specs
        if os.path.isfile(str(s.get("trajectory", "")))
        and os.path.isfile(str(s.get("labels", "")))
    ]
    return ok if len(ok) >= 6 else []


_SPECS = _golden_specs()
_skip = pytest.mark.skipif(
    not _SPECS,
    reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
)


# ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there, not cv2) ----
def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
    """The served litmus must pass a TYPED IndexingConfig (what production passes), not a dict ?
    `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError (#398). Build
    the config + drive the full served seam path on a synthetic trajectory (no golden data needed)."""
    from backend.config.models import IndexingConfig

```

```
        cfg = sga._served_idx_cfg(2)
        assert isinstance(cfg, IndexingConfig)
        assert cfg.fusion.min_crossings == 2
        traj = tmp_path / "synth.csv"
        rows = ["Frame,Visibility,X,Y"]
```

21. user (2026-07-02T02:49:02.731Z)

```
docs\ABLATION_LAYER.md:61:reportlab report (table of marginal contribution + CI + sign test + a
verdict column).
backend\eval\ablation.py:527:# The ~116-line reportlab PDF builder lives in a sibling module to
keep this file a pure driver;
backend\eval\ablation_pdf.py:1:"""reportlab PDF renderer for the ablation report (split out of
``backend.eval.ablation``).
backend\eval\ablation_pdf.py:4:booktabs-style PDF. ``reportlab`` is imported lazily inside
``export_pdf`` so importing this
backend\eval\ablation_pdf.py:8:Kept as its own module so the ~116-line reportlab builder doesn't
bloat the pure-driver
backend\eval\ablation_pdf.py:28:    """Render the ``AblationReport`` to a clean reportlab PDF: a
table of each signal's marginal
backend\eval\ablation_pdf.py:32:    Returns the written path. ``reportlab`` is an offline, pure-
Python dependency (no GPU)."""
backend\eval\ablation_pdf.py:33:    from reportlab.lib import colors
backend\eval\ablation_pdf.py:34:    from reportlab.lib.enums import TA_CENTER, TA_LEFT
backend\eval\ablation_pdf.py:35:    from reportlab.lib.pagesizes import A4
backend\eval\ablation_pdf.py:36:    from reportlab.lib.styles import ParagraphStyle,
getSampleStyleSheet
backend\eval\ablation_pdf.py:37:    from reportlab.lib.units import cm
backend\eval\ablation_pdf.py:38:    from reportlab.platypus import (
```

22. user (2026-07-02T02:49:31.699Z)

```
completed       failure refactor(D1/P10): extract orchestration.py from main.py (god-module
split, step 1)      CI      refactor/p10-split-main pull_request    28244991823     3m28s
2026-06-26T14:37:23Z
completed       failure refactor(D1/P10): extract orchestration.py from main.py (god-module
split, step 1)      CI      refactor/p10-split-main pull_request    28244470989     2m52s
2026-06-26T14:28:10Z
completed       failure chore: ruff format the whole repo (259 files, formatting-only)  CI
chore/ruff-format-repo  pull_request    28238584230     2m49s   2026-06-26T12:39:44Z
completed       failure feat(D13/P12): migrate ByteTrack to standalone trackers package CI
d13-p12/bytetrack-migration     pull_request    28237503425     2m57s   2026-06-26T12:18:03Z
completed       failure feat(D13/P12): migrate ByteTrack to standalone trackers package CI
d13-p12/bytetrack-migration     pull_request    28232820180     1m16s   2026-06-26T10:37:29Z
completed       failure feat(D13/P12): migrate ByteTrack to standalone trackers package CI
d13-p12/bytetrack-migration     pull_request    28232698842     1m20s   2026-06-26T10:34:46Z
completed       failure feat(D13/P12): migrate ByteTrack to standalone trackers package CI
d13-p12/bytetrack-migration     pull_request    28191091090     3m5s    2026-06-25T18:16:28Z
completed       failure chore: ruff format gemini_refine.py + trajectory_hybrid.py      CI
chore/format-gemini-refine-trajectory-hybrid    pull_request    28183805969     2m35s
2026-06-25T16:08:07Z
completed       failure fix(tests): resolve remaining test signature mismatches from app-factory
refactor       CI      fix/remaining-test-sigs pull_request    28093413101     2m47s
2026-06-24T10:53:35Z
completed       failure refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/?
CI      master  push    28092411029     2m34s   2026-06-24T10:33:47Z
completed       failure refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7)
CI      fix/app-factory-refactor-d2     pull_request    28091928133     2m52s
2026-06-24T10:24:39Z
completed       failure refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7)
CI      fix/app-factory-refactor-d2     pull_request    28091694811     2m47s
2026-06-24T10:20:11Z
completed       failure refactor: segmenters use typed AppConfig access (D3-b)  CI
refactor/segmenters-typed-config        pull_request    28060809741     2m28s
2026-06-23T22:16:04Z
```

```
completed       failure refactor: segmenters use typed AppConfig access (D3-b)  CI
refactor/segmenters-typed-config        pull_request    28059280286     2m23s
2026-06-23T21:46:22Z
completed       failure fix: segmenter_must_be_registered validator actually validates (B5)
CI      fix/segmenter-validator-actually-validates      pull_request    28052756302     2m43s
2026-06-23T19:51:33Z
---
```

23. user (2026-07-02T02:49:33.094Z)

```
config.json
---
af1ccbe chore(#399): launch-PR process hardening ? link-check CI, PR template, doc cross-links
(#402)
ffb1f53 ci: harden the 'System libs' apt step against the runner's Microsoft-repo 403 flake
(#391)
---
/c/Users/avidu/AppData/Local/Programs/Python/Python313/Scripts/ruff
/c/Users/avidu/AppData/Local/Programs/Python/Python313/Scripts/mypy
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
```

24. user (2026-07-02T02:49:33.112Z)

```
        assert saved["deployment"] == {"profile": "", "compute_target": ""}
        cfg = load_config(str(p))
        assert cfg.deployment.profile == ""


    def test_repo_config_json_has_no_profile():
        """The shipped config.json must not silently opt into a profile (default path stays
today's
        behaviour for the real deployment)."""
        repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
        cfg_path = os.path.join(repo_root, "config.json")
        if not os.path.exists(cfg_path):
            pytest.skip("no repo config.json")
        cfg = load_config(cfg_path)
        assert cfg.deployment.profile == ""
```

25. user (2026-07-02T02:49:33.273Z)

```
tests/test_reporting_harvest.py:14
tests/test_reporting_spec_faq.py:2
tests/test_reporting.py:5
```

26. user (2026-07-02T02:49:53.578Z)

```
All checks passed!
```

27. user (2026-07-02T02:49:53.975Z)

```
tests\test_served_gate_a_regression.py:5

Found 5 total occurrences across 1 file.
```

28. user (2026-07-02T02:50:12.974Z)

```
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage"  [attr-
```

```
defined]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Found 1 error in 1 file (checked 141 source files)
```

29. user (2026-07-02T02:50:34.697Z)

```
        self,
        *,
        client: Any = None,
        project: Optional[str] = None,
        signed_url_ttl: int = 3600,
        **_ignored,
    ) -> None:
        self._ttl = signed_url_ttl
        if client is not None:
            self._client = client
            return
        from google.cloud import (
            storage,  # type: ignore  # lazy: only when a gcs:// URI is used
        )

        self._client = storage.Client(project=project)

    def _blob(self, ref: StorageRef):
        return self._client.bucket(ref.bucket).blob(ref.key)

```

30. user (2026-07-02T02:50:49.151Z)

```
tests/test_yolo_hybrid.py::test_yolo_hybrid_store_candidates_disabled
tests/test_yolo_hybrid.py::test_yolo_chunk_dirs_follow_output_dir

1558 tests collected in 12.15s
```

31. user (2026-07-02T02:51:08.648Z)

```
    agg = aggregate([])
    assert agg["n"] == 0 and agg["f1"] is None


# --- W2: net-crossings gate (Gate-A) on the windowing --------------------------------
#
def test_windows_for_netcross_gate(tmp_path):
    pytest.importorskip(
        "cv2"
    )  # rally_gate import path tolerance (no cv2 actually needed)
    video = {
        "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
        "fps": 30.0,
        "frame_width": 1280.0,
    }
```

32. user (2026-07-02T02:51:11.155Z)

{"checks":[{"conclusion":"SUCCESS","name":"Ruff lint"},{"conclusion":"SUCCESS","name":"Leak scan (PII / attribution guard)"},{"conclusion":"SUCCESS","name":"Mypy (lenient baseline)"},{"conclusion":"SUCCESS","name":"Syntax sweep + import smoke (no GPU stack)"},{"conclusion":"FAILURE","name":"Full test suite (offline, mocked)"},{"conclusion":"SUCCESS","name":"Golden fixtures cross-OS (ubuntu-latest)"},{"conclusion":"SUCCESS","name":"Golden fixtures cross-OS (windows-latest)"},{"conclusion":"SUCCESS","name":"Golden fixtures cross-OS (macos-latest)"},{"conclusion":"SUCCESS","name":"Dependency license scan (no copyleft in runtime deps)"},{"conclusion":"SUCCESS","name":"Built wheel smoke (install +

launch)"}],"state":"OPEN","title":"refactor(D1/P10): extract orchestration.py from main.py (god-module split, step 1)"}

33. assistant (2026-07-02T02:52:23.429Z)

All checks complete. I have a full picture of the engine's CI + test health. Compiling the findings.

34. user (2026-07-02T02:53:14.185Z)

Structured output provided successfully